

AI Engineering Agentic Track: The Complete Agent & MCP Course

There has been different definitions on what an AI Agent really is. From OpenAI, to HuggingFace we have seen different approaches to define what an AI Agent is:

DEF

Sam Altman

"AI systems that can do work for you independently"

Early 2025 consensus (Anthropic, Hugging Face, etc)

"AI systems where an LLM controls the workflow"

Current prevailing view

"An LLM with Tools in a Loop to achieve a goal"

Agentic Systems

Anthropic distinguishes between two types:

- **Workflows** are systems where LLMs and tools are orchestrated through predefined code paths. Think of the Deep Research feature in several LLMs. There is a series of LLMs calls/Steps in a predefined path.
- **Agents** are dynamic systems where LLMs direct their own processes and tool usage, maintaining control over how they accomplish tasks.

NOTE

Sometimes in a corporate or production business system, you may lean more toward Workflows, because you want it to feel reproducible, and focus on completing specific tasks using a predictable path. This doesn't make it less of an Agentic System. But this is something more business will lean towards.

REMEMBER

This is from Anthropic's **Building Effective Agents** article:

When (and when not) to use agents

When building applications with LLMs, **p: we recommend finding the simplest solution possible**, and only increasing complexity when needed. This might mean not building agentic systems at all. **p: Agentic systems often trade latency and cost for better task performance, and you should consider when this tradeoff makes sense.**

When more complexity is warranted, **workflows** offer **predictability** and **consistency for well-defined tasks**, whereas agents are the better option when flexibility and model-driven decision-making are needed at scale. For many applications, however, optimizing single LLM calls with retrieval and in-context examples is usually enough.

Important gotcha's

- **Anthropomorphizing:** (stop treating agents like humans or give them human traits) a common red flag when agent architectures that assign human-like responsibilities to agents
- Build agent systems to solve problems, real business problems, in a measurable way. Don't do agents for the sake of needing or wanting to have one. Do not solutionize without having a strong business context; what is the business problem we are solving and does it really require an Agent/LLM
- Decide your agent architecture based on performance
- LLMs generate plausible and believable outputs, not necessarily accurate outputs. Aligning the outputs with business results? That's your job.

The Agent Landscape

- **Builders** for **Non-technical** people so they can **build** AI Agents (n8n, ElevenLabs, OpenAI Agent Builder, Crew AI Studio)
- **Products** for **Non-technical** people so they can **use** AI Agents (Claude Code, Cowork, OpenClaw, Claude Design)
- **Runtimes** for **Technical** people so they **execute** AI Agents (AWS Lambda, AWS Bedrock AgentCore, Claude Managed Agents, Vertex AI Agent Engine)
- **Frameworks** for **Technical** people so they can **develop** AI Agents (OpenAI Agents SDK, CrewAI, LangGraph/LangChain, Google ADK)

Agent Frameworks

[!Def]

Agent Frameworks

Helper code that makes it easier to create Agents based on LLM calls

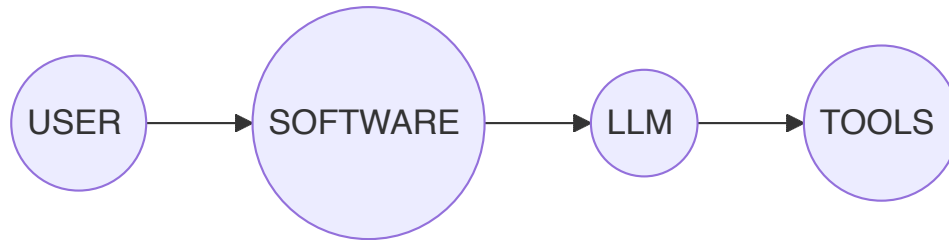
Orchestration: chaining together LLM calls

Tool calling: equipping an LLM with abilities, including with MCP

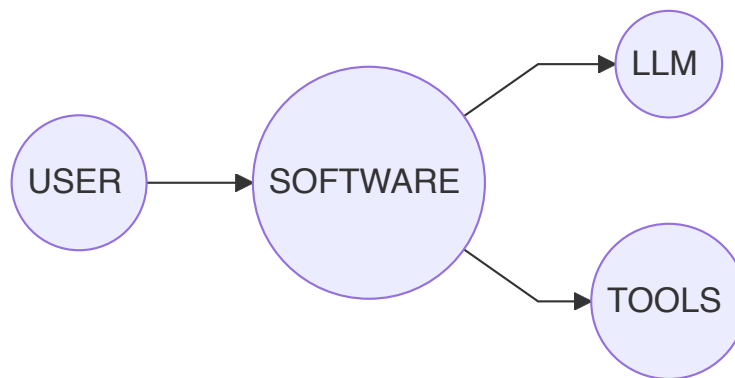
b:Examples: OpenAI Agents SDK, Google ADK, AWS Strands Agents

Tool Calling

People assume this process



In reality it is more like this:



[!DEF]

Agentic AI involves building a harness around LLMs, crafting the inputs and interpreting the outputs, in a loop, so that they can use **use tools** and **be autonomous**

Agent Frameworks are **helper code** that makes it faster to implement these techniques, but they're **not required**

Context Engineering

Referencing [The New Skill is Not Prompting, It's Context Engineering](#) post.

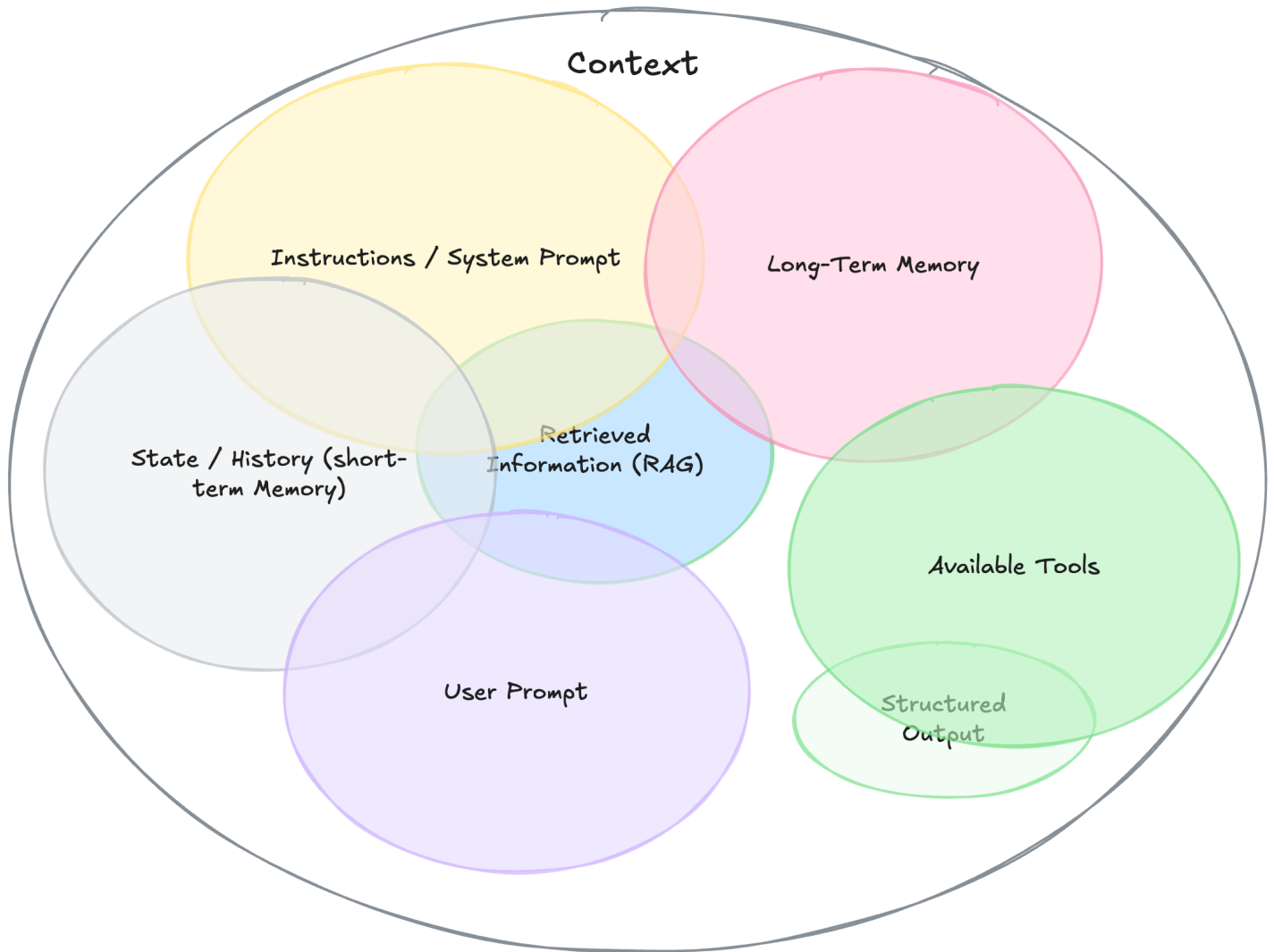
[!DEF]

“Context Engineering is the discipline of designing and building dynamic systems that provides the right information and tools, in the right format, at the right time, to give a LLM everything it needs to accomplish a task.”

“the art of providing all the context for the task to be plausibly solvable by the LLM.”

The secret to building truly effective AI agents has less to do with the complexity of the code you write, and everything to do with the **quality of the context you provide.**

Context Engineering



- **Instructions / system prompt:** An initial set of instructions that define the behavior of the model during a conversation, can/should include examples, rules
- **User Prompt:** Immediate task or question from the user.
- **Available Tools:** Definitions of all the functions or built-in tools it can call (e.g., `check_inventory`, `send_email`).
- **Structured Output:** Definitions on the format of the model's response, e.g. a JSON object.ÇÇÇ
- **State / History (shot-term memory):** The current conversation, including user and model responses that have led to this moment.
- **Long-term memory:** Persistent knowledge base, gathered across many prior conversations, containing learned user preferences, summaries of past projects, or facts it has been told to remember for future use.

- **Retrieved Information (RAG):** External, up-to-date knowledge, relevant information from documents, databases, or APIs to answer specific questions.

Async Python (**asyncio**)

- All the Agent Frameworks use **asynchronous python**
- All your methods and functions start **async**
- Anytime you call them, use **await**

```
1 async def do_some_processing():
2     # Do some work
3     return "done!"
4
5 result = await do_some_processing()
6
```

- Agentic LLMs means calling and doing multiple things parallel
- **Asyncio** provides a lightweight alternative to threading or multiprocessing
- Once we add **async** to a Python function, we don't call it a function anymore, instead we refer to it as **coroutines**, as they are special functions that can be paused and resumed.
- Calling a coroutine doesn't execute it immediately, it returns a coroutine object
- To actually run a coroutine, you must **await** it, which schedules it for execution within an **event loop**
 - The event loop is aware of all the coroutines that are awaiting, and only one coroutine is running at a time. It is a brute force manual way of scheduling coroutines.
 - While a coroutine is waiting (e.g. for I/O), the **event loop** can run other coroutines

Why Asyncio?

Yes, **asyncio** is incredibly useful for **Agentic AI** because AI agents spend most of their time waiting for network responses rather than processing heavy math on your local CPU.

When you call an LLM API (like OpenAI or Anthropic), your local system does almost zero computational work; it simply waits for the external server to stream or return the text tokens.

When to Use **asyncio**

You should use **asyncio** when your application is **I/O-bound** (Input/Output bound), meaning the execution speed is limited by data transmission rather than CPU processing power.

- **Network Requests:** Fetching data from multiple APIs, web scraping, or microservice communication.
 - **Database Operations:** Querying external databases or waiting for slow disk read/write actions.
 - **Chatbots & Web Servers:** Handling thousands of simultaneous user connections or long-lived WebSockets.
-

Why **asyncio** is Critical for Agentic AI

Agentic workflows inherently rely on sequential and parallel external tool calls. **asyncio** optimizes these workflows in four distinct ways:

1. Parallel Tool Calling (Function Calling)

When an LLM decides to call three different tools simultaneously (e.g., checking a weather API, searching Google, and reading a local file), standard synchronous code runs them one after another. **asyncio** lets you fire all three requests at the exact same moment.

2. Multi-Agent Collaborations

If you have a supervisor agent managing five worker agents, those worker agents can think, search, and generate responses at the same time. The supervisor waits for all of them concurrently, drastically cutting down total execution time.

3. Real-Time Token Streaming

When building a user interface for an agent, you want tokens to stream to the user in real time while the agent continues running background tasks (like logging data or predicting the next step). Async loops keep the UI responsive.

4. Handling High Volume User Traffic

If you host an AI agent platform where hundreds of users chat with agents simultaneously, standard threads consume massive memory. **asyncio** allows a single server thread to manage thousands of active agent sessions at a negligible memory cost.

Summary of Alternatives

Use Case	Best Tool	Why?
Calling LLM APIs / External Tools	<code>asyncio</code>	High network waiting time , low CPU usage .
Local Vector Search / Embeddings	<code>multiprocessing</code>	Heavy CPU/GPU matrix multiplication math .
Local File Operations / Logging	<code>threading</code> or <code>asyncio</code>	Mixed disk wait times .

`asyncio.gather` is the primary tool used to fire multiple asynchronous tasks simultaneously and wait for all of them to complete. It acts like a manager that launches multiple coroutines at once, monitors their progress, and collects their results into a single list

```

1  async def do_some_processing() → str:
2      # Do some work
3      return "done!"
4
5  # running the function returns aa coroutine
6  my_coroutine = do_some_processing()
7
8  # awaiting the coroutine returns a result
9  my_result = await my_coroutine
10
11 results = await asyncio.gather(
12     do_some_processing(),
13     do_other_processing(),
14     do_yet_more_processing()
15 )

```

How `asyncio.gather` Works Under the Hood

When you use `asyncio.gather` , it does not create threads or processes. Instead, it registers all the passed coroutines with the single-threaded event loop at the exact same time.

1. **Simultaneous Launch:** The event loop starts the first coroutine until it hits its first `await` (e.g., a network request).
2. **Immediate Switching:** While that first request is waiting for a network response, the loop instantly switches to the second coroutine, running it until it hits its `await` .
3. **Result Collection:** This pattern continues down the line. As soon as network data returns for any of the tasks, the loop resumes that specific task until it finishes.
4. **Final Return:** Once every task is completely done, `asyncio.gather` returns a list containing all the results in the exact order you passed them.

The Agentic AI Use Case: Parallel Tool Calling

In Agentic AI, `asyncio.gather` is most commonly used when an LLM determines it needs to pull information from multiple independent sources before making a decision.

Code Example

Here is how an agent executes a Google Search, a Database Lookup, and a Weather API call concurrently instead of sequentially:

```

1  import asyncio
2  import time
3
4  async def fetch_search_results(query: str):
5      print(f"🔍 Starting Google Search for: {query}")
6      await asyncio.sleep(2.5) # Simulating a slow network API call
7      print("✅ Google Search complete")
8      return "Found 3 relevant articles."
9
10 async def fetch_user_profile(user_id: int):
11     print(f"🗄️ Querying database for user: {user_id}")
12     await asyncio.sleep(1.0) # Simulating database latency
13     print("✅ Database query complete")
14     return {"name": "Alex", "tier": "premium"}
15
16 async def fetch_weather(location: str):
17     print(f"☀️ Fetching weather for: {location}")
18     await asyncio.sleep(1.5) # Simulating weather API latency
19     print("✅ Weather API complete")
20     return "Sunny, 22°C"
21
22 async def main():
23     start_time = time.time()
24
25     # Fire all three tools concurrently
26     results = await asyncio.gather(
27         fetch_search_results("Agentic AI trends"),
28         fetch_user_profile(9482),
29         fetch_weather("San Francisco")
30     )
31
32     # Unpack the ordered results
33     search_data, user_data, weather_data = results
34
35     print("\n🤖 Agent received all tool data:")
36     print(f"Results: {search_data}, {user_data}, {weather_data}")
37     print(f"🕒 Total execution time: {time.time() - start_time:.2f} seconds")
38

```



```

39 # Run the event loop
40 asyncio.run(main())
41

```

Critical Risks with `asyncio.gather`

While powerful, you must watch out for two main issues when using `gather`:

- **The Domino Error Effect:** By default, if *one* task in a `gather` block throws an unhandled exception, the entire `asyncio.gather` statement raises that exception immediately. The other tasks keep running in the background, but you lose easy access to their results.
- **Rate Limiting:** If you pass 100 API calls into `asyncio.gather`, it will fire all 100 instantly. This will often trigger rate limits (`429 Too Many Requests`) from LLM or data providers.

[!Hint]

Rule of Thumb: If your program spends 95% of its time waiting for a server or socket to reply, use `asyncio`. If it spends 95% of its time maxing out the CPU, use `multiprocessing`.

How It Works: Core Building Blocks

- **Event Loop:** The central engine that monitors registered tasks, switches between them when they pause, and handles I/O callbacks.
- **Coroutine (`async def`):** A function that returns a coroutine object instead of executing immediately when called.
- **`await`:** A pause point. It tells the event loop: *"Pause this function here until this operation finishes, and run other tasks in the meantime."*
- **Task (`asyncio.create_task`):** A wrapper that schedules a coroutine to run on the event loop concurrently in the background.

Python Review

Other Python concepts you need to master

 **Note**

The key idea

A generator is not mainly about avoiding a `for` loop. It is about **controlling when the next item is produced**.

OpenAI Agents SDK

- Lightweight and flexible
- Stays out of the way
- Makes common activities easy

Remember: Agents represent a particular LLM with system prompts and tools

How Agents can collaborate with OpenAI Agents SDK:

- Agents as Tools
- Handoffs

Three steps to create Agents in this SDK:

1. Create an instance of Agent
2. Use `with trace()` to track the agent
3. Call `Runner.run()` to run one "Application Level Turn"

The runner keeps looping until it reaches a stopping point

 Note

With Coding Agents: **Be the Boss**